

Concorrência e sincronização

Condição de corrida, semáforos, produtor-consumidor e monitores

Apostila da aula de 7 de outubro de 2026

Weslei Ferreira Santos

MATA58 · Sistemas Operacionais
Instituto de Computação / UFBA

Material-base: Prof. Maycon Leone M. Peixoto. Os programas citados estão no repositório da disciplina e podem ser compilados com gcc ou make.

Sumário

1	Antes de começar	4
1.1	O que você deve conseguir ao final	4
1.2	Como usar os exemplos	4
2	De processos a threads	5
2.1	O que é uma thread	5
2.2	Criando threads em C	5
2.3	Concorrência e paralelismo	6
2.4	Por que usar threads	6
2.5	Threads do usuário e threads do núcleo	7
2.6	A ordem de execução não é sua	7
3	Condição de corrida	7
3.1	contador++ não é uma operação só	7
3.2	O que o processador realmente executa	8
3.3	Uma intercalação que perde um incremento	8
3.4	Quantas intercalações existem	8
3.5	Um exemplo do mundo real: a conta bancária	9
3.6	Por que o erro é difícil de achar	9
3.7	O programa corrida.c	9
4	Região crítica e exclusão mútua	11
4.1	Definições	11
4.2	O que uma boa solução precisa garantir	11
4.3	Por que uma simples variável de trava não funciona	12
4.4	Desabilitar interrupções	12
4.5	Alternância estrita	12
4.6	Ajuda do hardware: instruções atômicas	13
4.7	Consertando o corrida.c	13
5	O algoritmo de Peterson	15
5.1	A ideia	15
5.2	Os dois cenários	15

5.3	Por que Peterson está correto	15
5.3.1	Exclusão mútua	15
5.3.2	Progresso	16
5.3.3	Espera limitada	16
5.4	Peterson em C	16
5.4.1	Por que atomics	17
5.4.2	O que dá errado sem a ordem garantida	17
5.5	Espera ocupada	18
5.6	Quando girar compensa	18
6	Semáforos	19
6.1	Dormir e acordar: o despertar perdido	19
6.2	Definição	19
6.3	Como o semáforo funciona por dentro	20
6.4	Semáforo como mutex	20
6.5	Mutex e semáforo binário não são idênticos	20
6.6	Funções POSIX	21
7	O problema do produtor-consumidor	21
7.1	O problema	21
7.2	Os três semáforos	21
7.3	Os passos de cada thread	22
7.4	Simulação com $N = 3$	22
7.5	O programa prod_cons.c	22
7.6	Por que a ordem dos sem_wait importa	25
7.7	Vários produtores e vários consumidores	25
8	Deadlock, inanição e livelock	26
8.1	As quatro condições do deadlock	26
8.2	Como evitar na prática	26
8.3	Inanição e livelock	26
9	Monitores em Java	27
9.1	Por que monitores	27
9.2	Origem: Hoare e Brinch Hansen	27
9.3	As ferramentas do Java	28
9.4	Mailbox 3b: sem sincronização	28
9.5	Mailbox 3s: com monitor	30

9.6	Por que while e não if	31
9.7	Semântica de Hoare e semântica de Mesa	31
9.8	O limite da mailbox 3s	32
9.9	Uma caixa de correio correta, em Java	32
9.10	Ferramentas prontas do Java	33
9.11	Semáforos ou monitores?	33
10	Demonstração visual: Produção de dinheiro	34
10.1	A simulação	34
10.2	Quatro experimentos	34
10.2.1	Experimento 1: ritmos iguais	34
10.2.2	Experimento 2: produtor rápido	34
10.2.3	Experimento 3: consumidor rápido	34
10.2.4	Experimento 4: a volta do buffer e a pausa	34
11	Atividade prática	35
11.1	O que fazer	35
11.2	Compilar e testar	35
11.3	Se der errado	36
11.4	Perguntas para responder em dupla	36
12	Resumo	36
13	Exercícios de fixação	37
	Para saber mais	39

1 Antes de começar

Na aula de 5/10 vimos que cada processo tem a sua própria memória: o pai somou 0 porque as escritas dos filhos ficaram nas cópias deles, e para levar um resultado de um processo a outro foi preciso um pipe. Nesta aula o problema se inverte. As **threads** de um mesmo processo dividem a memória. Comunicar fica fácil; o difícil passa a ser impedir que uma thread atrapalhe a outra.

Se as threads compartilham a memória, como impedir que uma atrapalhe a outra?

O caminho da aula é este: primeiro vemos o problema acontecer (a **condição de corrida**); depois damos nome ao trecho perigoso (**região crítica**) e estudamos três ferramentas para protegê-lo: o algoritmo de **Peterson**, os **semáforos** e os **monitores**. No meio, o problema clássico do **produtor-consumidor** mostra para que serve cada ferramenta, e uma simulação visual deixa tudo isso visível.

1.1 O que você deve conseguir ao final

- ▶ Explicar por que `contador++` em duas threads perde incrementos.
- ▶ Definir região crítica e exclusão mútua.
- ▶ Explicar Peterson e o que é espera ocupada.
- ▶ Dizer o que `mutex`, `empty` e `full` controlam no produtor-consumidor e por que a ordem dos `sem_wait` importa.
- ▶ Explicar `synchronized`, `wait` e `notify` em Java, e por que se usa `while` antes de `wait`.
- ▶ Completar o produtor-consumidor com os semáforos certos.

1.2 Como usar os exemplos

Na raiz do repositório:

```
$ gcc -Wall -Wextra examples/dia07/corrida.c -o corrida -pthread && ./corrida
$ gcc -Wall -Wextra examples/dia07/corrida_atomic.c -o corrida-atomic -pthread && ./corrida-atomic
$ gcc -Wall -Wextra examples/dia07/corrida_mutex.c -o corrida-mutex -pthread && ./corrida-mutex
$ gcc -Wall -Wextra examples/dia07/peterson-code.c -o peterson -pthread && ./peterson
$ gcc -Wall -Wextra examples/dia07/prod_cons.c -o prod_cons -pthread && ./prod_cons
$ make java && timeout 5 java -cp build/java-3b ThreadSync3b
$ make java && timeout 5 java -cp build/java-3s ThreadSync3s
$ make run-caixa # caixa de correio correta em Java
$ make run-simulation # janela da simulacao Producao de dinheiro
```

A opção `-pthread` prepara a compilação para usar threads. No Linux atual o programa costuma compilar mesmo sem ela, mas em sistemas mais antigos aparece `undefined reference to pthread_create`;

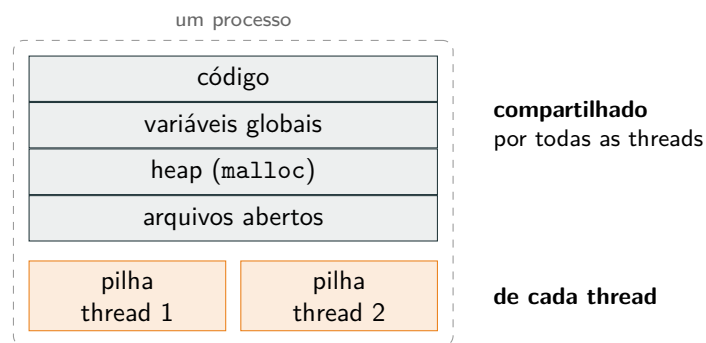
use sempre. As mailboxes Java rodam para sempre: `timeout 5` as encerra depois de 5 segundos (ou use `Ctrl + C`).

2 De processos a threads

2.1 O que é uma thread

Definição

Uma **thread** é um fluxo de execução dentro de um processo. Um processo pode ter várias threads; todas executam o mesmo programa e enxergam a mesma memória, mas cada uma tem o seu próprio ponto de execução, os seus registradores e a sua pilha.



Compartilhado pelas threads	Próprio de cada thread
Código do programa	Pilha (variáveis locais, chamadas de função)
Variáveis globais e static	Registradores e contador de programa
Heap (o que foi alocado com malloc)	Estado (executando, pronta, bloqueada)
Arquivos e pipes abertos	

2.2 Criando threads em C

Com a biblioteca POSIX threads (pthread):

```
#include <pthread.h>

void *somar(void *arg) { ...; return NULL; } /* o que a thread executa */

pthread_t t1, t2;
pthread_create(&t1, NULL, somar, NULL);      /* cria; a thread já começa a rodar */
pthread_create(&t2, NULL, somar, NULL);
pthread_join(t1, NULL);                     /* espera t1 terminar */
pthread_join(t2, NULL);
```

- ▶ `pthread_create` recebe o endereço onde guardar o identificador, atributos (`NULL` = padrão), a função que a thread vai executar e um argumento para ela.
- ▶ `pthread_join` faz quem chamou esperar a thread terminar. É o equivalente do `waitpid` para threads.
- ▶ Ao contrário das funções que vimos na aula passada, as funções `pthread_*` não devolvem `-1`: devolvem 0 em sucesso ou o **código do erro**.

	Processos (5/10)	Threads (7/10)
Criar	<code>fork()</code>	<code>pthread_create()</code>
Esperar terminar	<code>waitpid()</code>	<code>pthread_join()</code>
Memória	cópia separada	a mesma
Comunicar	pipe	variável compartilhada
Risco principal	dados não chegam	dados se atropelam

2.3 Concorrência e paralelismo

Duas palavras que costumam ser confundidas:

Definição

Concorrência: várias tarefas estão em andamento no mesmo período de tempo, e a execução delas se **intercala**. Basta uma CPU: o sistema alterna entre elas tão rápido que parecem simultâneas.

Paralelismo: várias tarefas executam **literalmente ao mesmo tempo**, em núcleos diferentes.

Todo programa paralelo é concorrente, mas não o contrário. Os problemas desta aula aparecem nos dois casos: numa CPU única, a troca de thread pode acontecer no meio de uma operação; com vários núcleos, as duas threads podem até executar a mesma linha no mesmo instante. O computador usado para testar os exemplos tinha 12 núcleos.

2.4 Por que usar threads

- ▶ **Responsividade:** um programa com interface gráfica pode continuar respondendo ao usuário enquanto uma thread faz um cálculo demorado. A simulação desta aula faz exatamente isso.
- ▶ **Compartilhamento:** as threads já enxergam os mesmos dados; não é preciso pipe nem cópia.
- ▶ **Economia:** criar uma thread é bem mais barato do que criar um processo, porque não é preciso montar um novo espaço de endereçamento nem copiar tabelas. Trocar de uma thread para outra do mesmo processo também é mais barato.
- ▶ **Aproveitar vários núcleos:** um processo com uma thread só usa um núcleo de cada vez; com várias, pode usar vários.

2.5 Threads do usuário e threads do núcleo

Uma thread pode ser gerenciada de dois jeitos:

- **Threads do usuário:** uma biblioteca, dentro do processo, cria e alterna as threads. O núcleo do SO só enxerga um processo. É rápido, mas se uma thread fizer uma chamada de sistema bloqueante, todas param, e não há como usar vários núcleos.
- **Threads do núcleo:** o próprio SO conhece cada thread e a escalona como se fosse um processo leve. Uma thread bloqueada não trava as outras, e threads diferentes podem rodar em núcleos diferentes.

Os modelos que combinam os dois são chamados **N:1** (várias threads do usuário sobre uma do núcleo), **1:1** (cada thread do usuário é uma thread do núcleo) e **M:N** (várias sobre várias). No Linux, a biblioteca pthread (NPTL) usa o modelo **1:1**: cada pthread_create cria uma tarefa no núcleo pela chamada de sistema clone(), a mesma família do fork(), só que pedindo para compartilhar a memória em vez de copiá-la. As threads do Java tradicional também são 1:1; a partir do Java 21 existem também as *threads virtuais*, no modelo M:N.

2.6 A ordem de execução não é sua

O escalonador do Linux é **preemptivo**: ele pode tirar uma thread da CPU a qualquer momento, por exemplo quando acaba a sua fatia de tempo ou quando chega uma interrupção. Não há como o programa saber em qual instrução isso vai acontecer. Por isso, quando duas threads usam o mesmo dado, precisamos raciocinar sobre **todas** as ordens possíveis, não só sobre a mais provável.

3 Condição de corrida

3.1 contador++ não é uma operação só

Para nós, contador++ parece uma coisa única. Para o processador, são três passos:

1. **ler** o valor da memória para um registrador;
2. **somar 1** no registrador;
3. **escrever** o registrador de volta na memória.

Entre um passo e outro, o escalonador pode tirar a thread da CPU e colocar outra. Num computador com vários núcleos, as duas threads podem até executar esses passos *ao mesmo tempo*.

3.2 O que o processador realmente executa

Compilando o `corrida.c` sem otimização (`gcc -O0 -S`), a linha `contador = contador + 1` vira três instruções de máquina (x86-64):

```
movl  contador(%rip), %eax    # 1. le contador da memoria para o registrador eax
addl  $1, %eax               # 2. soma 1 no registrador
movl  %eax, contador(%rip)   # 3. escreve eax de volta na memoria
```

Com otimização (`-O2`), o gcc gera uma instrução só:

```
addl  $1, contador(%rip)     # soma 1 direto na memoria
```

E mesmo assim os incrementos se perdem. Uma instrução só **não** é o mesmo que uma operação atômica: por dentro, o processador ainda lê o valor, soma e escreve, e outro núcleo pode ler o valor antigo no meio. Para que essa soma seja indivisível entre núcleos, é preciso o prefixo `lock` (`lock addl`), que é justamente o que o compilador gera quando usamos `atomic_fetch_add` (Seção 4.6).

3.3 Uma intercalação que perde um incremento

Contador começa em 0 e cada thread quer somar 1. O esperado é 2.

Passo	Thread A	Thread B	contador na memória
1	lê 0		0
2		lê 0	0
3	calcula 1		0
4		calcula 1	0
5	escreve 1		1
6		escreve 1	1

As duas threads leram 0 antes de qualquer uma escrever. A escrita de B apagou a de A, e um incremento se perdeu.

Condição de corrida

Situação em que o resultado de um programa depende da **ordem** em que as threads foram intercaladas. Como essa ordem muda a cada execução, o resultado também muda, e o erro pode aparecer só de vez em quando.

3.4 Quantas intercalações existem

Cada thread executa três passos em ordem (ler, somar, escrever), mas os passos de uma podem se encaixar de qualquer jeito entre os da outra. O número de intercalações possíveis é o número de maneiras de escolher, entre 6 posições, as 3 da thread A:

$$\binom{6}{3} = \frac{6!}{3!3!} = 20.$$

O resultado só sai correto (2) quando uma thread lê o valor *depois* que a outra escreveu, e isso exige que uma delas faça os três passos inteiros antes de a outra começar. Das 20 intercalações, só 2 dão certo (AAABBB e BBAAAA); as outras 18 perdem um incremento.

Na prática, as trocas de thread não acontecem a cada instrução: uma thread costuma executar milhares de voltas do laço antes de ser interrompida. Por isso nem toda volta perde um incremento. Mas com um milhão de voltas por thread, e com as duas rodando em núcleos diferentes ao mesmo tempo, as janelas ruins aparecem centenas de milhares de vezes.

3.5 Um exemplo do mundo real: a conta bancária

Imagine uma conta com R\$ 100 e duas operações de saque de R\$ 80 feitas ao mesmo tempo, cada uma por uma thread:

```
if (saldo >= valor)      /* 1. confere se ha saldo      */
    saldo = saldo - valor; /* 2. desconta              */
```

Se as duas threads executam o passo 1 antes de qualquer uma executar o passo 2, as duas veem saldo = 100, as duas aprovam o saque e a conta termina com R\$ -60, ou com R\$ 20 se uma escrita apagar a outra. O problema não está em nenhuma linha isolada: está no fato de que “conferir e agir” precisa acontecer **sem interrupção**. Esse padrão, chamado *check-then-act*, é uma das formas mais comuns de condição de corrida.

3.6 Por que o erro é difícil de achar

- ▶ Ele é **intermitente**: depende da ordem das threads, que muda com a carga da máquina, o número de núcleos e até com a presença de um `printf` de depuração.
- ▶ Um teste que passou mil vezes **não prova** que o programa está certo; prova só que a intercalação ruim não apareceu nessas mil vezes.
- ▶ Às vezes o erro some quando o programa é executado no depurador, porque o depurador muda os tempos. Esse tipo de erro ganhou o apelido de *heisenbug*.

Condições de corrida já causaram acidentes reais. A mais citada é a da máquina de radioterapia Therac-25 (1985 a 1987): uma corrida entre a tarefa que lia o teclado do operador e a que configurava o feixe fez a máquina aplicar doses muito acima do previsto em alguns pacientes.

3.7 O programa `corrida.c`

```
1  /* Aula de 7/10: condicao de corrida com duas threads e um contador comum.
2  *
3  * O que observar:
4  *   1. Duas threads somam 1 ao MESMO contador, ITERACOES vezes cada uma.
5  *   2. O esperado seria 2 * ITERACOES, mas o resultado costuma sair MENOR
6  *      e muda a cada execucao.
7  *   3. contador++ nao e uma operacao unica: e LER, SOMAR 1 e ESCRIVER.
8  *      Se as duas threads leem o mesmo valor antes de escrever, um
9  *      incremento se perde.
10 *
11 * Atencao: acessar uma variavel comum de duas threads sem sincronizacao e
12 * uma "data race", e em C isso e comportamento indefinido. Este programa
```

```

13  * existe so para MOSTRAR o problema; nunca escreva codigo assim de verdade.
14  * O volatile abaixo nao conserta nada: ele so impede o compilador de juntar
15  * as somas numa conta unica, para que a corrida apareca na execucao.
16  *
17  * Compile e execute:
18  * gcc -Wall -Wextra examples/dia07/corrída.c -o corrida -pthread
19  * ./corrída
20  */
21  #include <pthread.h>    /* pthread_create, pthread_join */
22  #include <stdio.h>      /* printf, fprintf */
23  #include <stdlib.h>     /* EXIT_SUCCESS, EXIT_FAILURE */
24  #include <string.h>     /* strerror */
25
26  #define ITERACOES 1000000
27
28  /* Variavel global: TODAS as threads do processo enxergam a mesma.
29   * (Na aula de 5/10, cada processo tinha a sua copia; aqui nao.) */
30  static volatile int contador = 0;
31
32  /*Codigo executado por cada thread. */
33  static void *somar(void *arg)
34  {
35      (void)arg; /* nao usamos o argumento */
36      for (int i = 0; i < ITERACOES; ++i) {
37          /* REGIAO CRITICA SEM PROTECAO: le, soma 1 e escreve.
38           * A outra thread pode ler o valor antigo no meio desse caminho. */
39          contador = contador + 1;
40      }
41      return NULL;
42  }
43
44  int main(void)
45  {
46      pthread_t t1, t2;
47
48      /* Cria as duas threads. Elas comecam a executar somar() imediatamente
49       * e disputam a CPU entre si (e com a main). */
50      int erro = pthread_create(&t1, NULL, somar, NULL);
51      if (erro == 0) erro = pthread_create(&t2, NULL, somar, NULL);
52      if (erro != 0) {
53          fprintf(stderr, "pthread_create: %s\n", strerror(erro));
54          return EXIT_FAILURE;
55      }
56
57      /* pthread_join espera cada thread terminar (o "waitpid" das threads). */
58      pthread_join(t1, NULL);
59      pthread_join(t2, NULL);
60
61      printf("Contador: %d (esperado: %d)\n", contador, 2 * ITERACOES);
62      if (contador != 2 * ITERACOES)
63          printf("Perdemos %d incrementos por causa da condicao de corrida.\n",
64                2 * ITERACOES - contador);
65      return EXIT_SUCCESS;
66  }

```

Algumas execuções reais:

```

$ ./corrída
Contador: 1006933 (esperado: 2000000)
Perdemos 993067 incrementos por causa da condicao de corrida.
$ ./corrída
Contador: 1349235 (esperado: 2000000)

```

```
Perdemos 650765 incrementos por causa da condicao de corrida.  
$ ./corrida  
Contador: 1022204 (esperado: 2000000)  
Perdemos 977796 incrementos por causa da condicao de corrida.
```

Quase metade dos incrementos se perde, e o número é diferente a cada vez.

Data race e comportamento indefinido

Em C, duas threads acessando a mesma variável sem sincronização, com pelo menos uma escrevendo, é uma **data race**, e o padrão diz que o comportamento é **indefinido**. Não é correto afirmar que “o resultado só pode ser um pouco menor”. O `volatile` do programa não conserta nada: ele só impede o compilador de juntar o milhão de somas numa conta única, para que a corrida apareça. Acrescentar `sleep` também não conserta: só muda a probabilidade.

4 Região crítica e exclusão mútua

4.1 Definições

Definição

A **região crítica** é o trecho do código que acessa um recurso compartilhado (no `corrida.c`, a linha `contador = contador + 1`).

A **exclusão mútua** é a garantia de que no máximo **uma** thread por vez esteja dentro da região crítica.

Se só uma thread por vez pode executar “ler, somar, escrever”, a intercalação da Seção 3.3 se torna impossível e nenhum incremento se perde.

4.2 O que uma boa solução precisa garantir

Tanenbaum lista quatro condições:

1. Duas threads nunca estão ao mesmo tempo na região crítica.
2. A solução não depende da velocidade das threads nem do número de CPUs.
3. Uma thread fora da região crítica não pode impedir outra de entrar.
4. Nenhuma thread espera para sempre para entrar.

4.3 Por que uma simples variável de trava não funciona

Uma ideia ingênua é usar uma variável trava: 0 significa livre, 1 significa ocupado.

```
while (trava == 1) { } /* espera ficar livre */
trava = 1;             /* ocupa */
/* regiao critica */
trava = 0;             /* libera */
```

O problema é o mesmo do contador++: entre *ver* que a trava está livre e *escrever* 1 nela, a outra thread pode fazer o mesmo teste. As duas veem 0, as duas escrevem 1 e as duas entram. A própria trava sofre condição de corrida. Precisamos de um protocolo mais cuidadoso, ou de ajuda do hardware e do sistema operacional.

4.4 Desabilitar interrupções

Numa máquina com **uma** CPU, a thread só perde o processador por causa de uma interrupção (do relógio, de um dispositivo). Se ela desabilitar as interrupções antes da região crítica e reabilitar depois, ninguém a interrompe. Essa solução tem dois problemas graves:

- Só o **núcleo do SO** pode desabilitar interrupções. Dar esse poder a um programa de usuário permitiria que ele travasse a máquina inteira.
- Com **vários núcleos**, desabilitar as interrupções de um núcleo não impede que outro núcleo execute a mesma região crítica.

O próprio núcleo ainda usa essa técnica por instantes muito curtos, mas ela não serve para programas comuns.

4.5 Alternância estrita

Outra ideia é uma variável vez que diz de quem é a vez de entrar:

```
/* thread 0 */
while (vez != 0) { }
/* regiao critica */
vez = 1;

/* thread 1 */
while (vez != 1) { }
/* regiao critica */
vez = 0;
```

Ela garante a exclusão mútua, mas **viola a condição 3**: as threads são obrigadas a se alternar. Se a thread 0 quer entrar duas vezes seguidas e a thread 1 está ocupada com outra coisa, fora da região crítica, a thread 0 fica esperando por uma thread que não está nem tentando entrar. Peterson resolve exatamente esse defeito, combinando vez com as bandeiras interessado.

4.6 Ajuda do hardware: instruções atômicas

Os processadores modernos oferecem instruções que leem e escrevem uma posição de memória **numa operação indivisível**, mesmo com vários núcleos:

Instrução	O que faz, de forma indivisível
<i>test-and-set</i> (TSL)	Lê o valor antigo e escreve 1. Se o antigo era 0, a trava estava livre e agora é minha.
<i>exchange</i> (xchg)	Troca o conteúdo de um registrador com o de uma posição de memória.
<i>compare-and-swap</i> (cmpxchg)	Se a memória ainda tem o valor esperado, troca pelo novo; senão, não mexe e avisa.
<i>fetch-and-add</i> (lock xadd)	Soma um valor e devolve o antigo.

Com *test-and-set*, a variável de trava da seção anterior passa a funcionar, porque “ver que está livre” e “ocupar” viram um passo só. Em C11, isso é o `atomic_flag`:

```
static atomic_flag ocupado = ATOMIC_FLAG_INIT;

void entrar(void) { while (atomic_flag_test_and_set(&ocupado)) { } } /* gira */
void sair(void)   { atomic_flag_clear(&ocupado); }
```

Uma trava assim, em que quem espera fica girando, se chama **spinlock**. Ela funciona para qualquer número de threads (Peterson, só para duas), mas ainda é espera ocupada.

4.7 Consertando o corrida.c

Há três jeitos de consertar o `corrida.c`, todos testados no repositório:

Conserto	Como funciona	Tempo medido
<code>atomic_fetch_add</code>	A própria soma vira indivisível (<code>lock addl</code>). Não há região crítica: há uma operação atômica.	0,02 s
<code>pthread_mutex_t</code>	Região crítica com trava. Quem encontra a trava ocupada dorme.	0,10 s
<code>spinlock (atomic_flag)</code>	Região crítica com trava. Quem encontra a trava ocupada gira.	0,10 s

Os três dão sempre 2000000. Os tempos são de uma execução na máquina de teste (12 núcleos), para 2 milhões de incrementos; servem só para comparar a ordem de grandeza. A operação atômica é a mais rápida porque cada soma é uma instrução só; as travas têm o custo extra de entrar e sair. Quando a região crítica é uma única operação simples, a operação atômica é a escolha natural; quando a região crítica tem várias linhas, como inserir num buffer, é preciso uma trava.

```
1 #include <pthread.h>
2 #include <stdatomic.h>
3 #include <stdio.h>
4 #include <stdlib.h>
5
6 #define ITERACOES 1000000
```

```

7
8 static atomic_int contador = 0;      /* agora e um inteiro atômico */
9
10 static void *somar(void *arg)
11 {
12     (void)arg;
13     for (int i = 0; i < ITERACOES; ++i)
14         atomic_fetch_add(&contador, 1);    /* soma indivisível */
15     return NULL;
16 }
17
18 int main(void)
19 {
20     pthread_t t1, t2;
21     if (pthread_create(&t1, NULL, somar, NULL) != 0 ||
22         pthread_create(&t2, NULL, somar, NULL) != 0) {
23         fprintf(stderr, "pthread_create falhou\n");
24         return EXIT_FAILURE;
25     }
26     pthread_join(t1, NULL);
27     pthread_join(t2, NULL);
28     printf("Contador: %d (esperado: %d)\n", atomic_load(&contador), 2 * ITERACOES);
29     return atomic_load(&contador) == 2 * ITERACOES ? EXIT_SUCCESS : EXIT_FAILURE;
30 }

```

```

1 #include <pthread.h>
2 #include <stdio.h>
3 #include <stdlib.h>
4
5 #define ITERACOES 1000000
6
7 static int contador = 0;                /* inteiro comum */
8 static pthread_mutex_t trava = PTHREAD_MUTEX_INITIALIZER; /* a "chave" */
9
10 static void *somar(void *arg)
11 {
12     (void)arg;
13     for (int i = 0; i < ITERACOES; ++i) {
14         pthread_mutex_lock(&trava);      /* entra: pega a chave (ou dorme) */
15         contador = contador + 1;          /* região crítica protegida */
16         pthread_mutex_unlock(&trava);     /* sai: devolve a chave */
17     }
18     return NULL;
19 }
20
21 int main(void)
22 {
23     pthread_t t1, t2;
24     if (pthread_create(&t1, NULL, somar, NULL) != 0 ||
25         pthread_create(&t2, NULL, somar, NULL) != 0) {
26         fprintf(stderr, "pthread_create falhou\n");
27         return EXIT_FAILURE;
28     }
29     pthread_join(t1, NULL);
30     pthread_join(t2, NULL);
31     printf("Contador: %d (esperado: %d)\n", contador, 2 * ITERACOES);
32     return contador == 2 * ITERACOES ? EXIT_SUCCESS : EXIT_FAILURE;
33 }

```

5 O algoritmo de Peterson

5.1 A ideia

Peterson resolve a exclusão mútua para **duas** threads usando só leituras e escritas comuns na memória (com a ordem garantida). Cada thread tem uma bandeira `interessado` e existe uma variável `vez` compartilhada:

```
/* entrada */
interessado[eu] = verdadeiro           "quero entrar"
vez = outro                           "mas, se houver empate, você primeiro"
enquanto interessado[outro] e vez == outro:
    esperar

/* região crítica */

/* saída */
interessado[eu] = falso                "saí, não quero mais"
```

5.2 Os dois cenários

Situação	O que acontece
O outro não quer entrar	<code>interessado[outro]</code> é falso: a condição do <code>enquanto</code> falha e eu entro direto. Não há alternância forçada.
Os dois querem entrar ao mesmo tempo	Os dois escrevem em <code>vez</code> . Só um valor fica: o da thread que escreveu por último . Essa thread vê <code>vez == outro</code> e espera; a outra entra.

Quando a thread que entrou sai, ela faz `interessado = falso`, e a que estava esperando passa.

5.3 Por que Peterson está correto

Peterson cumpre as condições da Seção 4.2. Um argumento informal:

Exclusão mútua. Suponha, por absurdo, que as duas threads estão na região crítica ao mesmo tempo. Então as duas escreveram `interessado = verdadeiro` e as duas escreveram em `vez`. A variável `vez` guarda um único valor: o da thread que escreveu por último, digamos a thread 1, que escreveu `vez = 0`. Quando a thread 1 testou o `enquanto`, viu `interessado[0]` verdadeiro e `vez == 0`, ou seja, a condição verdadeira, e teria de esperar. Ela só sairia do laço se a thread 0 mudasse `interessado[0]` para falso, o que só acontece quando a thread 0 **sai** da região crítica. Contradição: as duas não podem estar dentro ao mesmo tempo.

Progresso. Se só uma thread quer entrar, interessado[outro] é falso e ela entra sem esperar. Uma thread que não está interessada não atrapalha ninguém: é a condição 3, que a alternância estrita violava.

Espera limitada. Se as duas querem entrar, a que esperou entra assim que a outra sair, porque a outra, ao tentar entrar de novo, escreve vez = outro e cede a vez. Nenhuma thread pode passar na frente da outra duas vezes seguidas enquanto a outra está esperando.

5.4 Peterson em C

```

1  /* Peterson para duas threads: atomics C11 sequencialmente consistentes.
2  * Espera ocupada intencional para fins didaticos; nao ha alternancia forçada.
3  *
4  * O que observar:
5  * 1. Duas threads incrementam o MESMO contador, como em corrida.c,
6  *    mas agora cada incremento fica dentro de uma regioao critica.
7  * 2. enter_region / leave_region garantem que so UMA thread por vez
8  *    execute a regioao critica (exclusao mutua).
9  * 3. O resultado sai sempre 20000: nenhum incremento se perde.
10 * 4. Quem espera fica girando no while (espera ocupada): gasta CPU.
11 *
12 * Protocolo de Peterson (para a thread "eu"):
13 *   interested[eu] = verdadeiro      "quero entrar"
14 *   turn = outro                    "mas, se houver disputa, voce primeiro"
15 *   espera enquanto o outro quer entrar E a vez e dele
16 *   ... regioao critica ...
17 *   interested[eu] = falso          "sai, nao quero mais"
18 *
19 * Por que atomics? Com variaveis comuns, o compilador e o processador podem
20 * reordenar leituras e escritas, e o algoritmo deixa de funcionar. As
21 * operacoes atomic_load e atomic_store garantem a ordem que Peterson exige.
22 */
23 #include <pthread.h>    /* pthread_create, pthread_join */
24 #include <stdatomic.h>  /* atomic_int, atomic_bool, atomic_load, atomic_store */
25 #include <stdio.h>      /* printf, fprintf */
26 #include <stdlib.h>     /* exit, EXIT_SUCCESS, EXIT_FAILURE */
27 #include <string.h>     /* strcpy, strerror */
28
29 #define ITERACOES 10000
30 static atomic_int turn = 0;          /* de quem e a vez quando ambos querem entrar */
31 static atomic_bool interested[2];   /* interested[i]: a thread i quer entrar */
32 static char cr[16];                /* recurso compartilhado: nome de quem esta dentro */
33 static int contador;                /* recurso compartilhado: protegido pela regioao critica
34                                     */
35
36 /* Protocolo de ENTRADA: so retorna quando for seguro entrar. */
37 static void enter_region(int eu)
38 {
39     int outro = 1 - eu;             /* a outra thread: 0 <-> 1 */
40     atomic_store(&interested[eu], 1); /* anuncia: eu quero entrar */
41     atomic_store(&turn, outro);       /* cede a vez em caso de empate */
42     /* Espera enquanto o outro quer entrar E a vez e dele.
43      * Se o outro nao quer entrar, a condicao e falsa e eu entro direto.
44      * Se ambos querem, quem escreveu turn por ULTIMO e quem espera. */
45     while (atomic_load(&interested[outro]) && atomic_load(&turn) == outro) {
46         /* Espera ocupada: a thread continua rodando e testando, gastando CPU. */
47     }
48 }

```

```

49  /* Protocolo de SAIDA: avisa que nao esta mais na regio critica. */
50  static void leave_region(int eu)
51  {
52      atomic_store(&interested[eu], 0);
53  }
54
55  /*Codigo de cada thread: ITERACOES entradas e saidas da regio critica. */
56  static void *thread(void *arg)
57  {
58      int eu = *(int *)arg; /* 0 ou 1: qual das duas threads sou eu */
59      for (int i = 0; i < ITERACOES; ++i) {
60          enter_region(eu);
61          /* ===== REGIAO CRITICA: so uma thread por vez chega aqui ===== */
62          strcpy(cr, eu == 0 ? "thread0" : "thread1#####");
63          ++contador; /* agora o incremento nao se perde */
64          if (i == 0) printf("Regiao critica: %s\n", cr);
65          /* ===== fim da regio critica ===== */
66          leave_region(eu);
67      }
68      return NULL;
69  }
70
71  /* pthread_* devolvem o codigo de erro (0 = sucesso), e nao -1 com errno. */
72  static void check(int erro, const char *operacao)
73  {
74      if (erro != 0) {
75          fprintf(stderr, "%s: %s\n", operacao, strerror(erro));
76          exit(EXIT_FAILURE);
77      }
78  }
79
80  int main(void)
81  {
82      pthread_t threads[2];
83      int ids[2] = {0, 1}; /* cada thread recebe o endereco do seu numero */
84      atomic_init(&interested[0], 0);
85      atomic_init(&interested[1], 0);
86      for (int i = 0; i < 2; ++i)
87          check(pthread_create(&threads[i], NULL, thread, &ids[i]), "pthread_create");
88      /* Espera as duas threads terminarem antes de ler o contador. */
89      for (int i = 0; i < 2; ++i)
90          check(pthread_join(threads[i], NULL), "pthread_join");
91      printf("Contador: %d (esperado: %d)\n", contador, 2 * ITERACOES);
92      return contador == 2 * ITERACOES ? EXIT_SUCCESS : EXIT_FAILURE;
93  }

```

```

$ ./peterson
Regiao critica: thread0
Regiao critica: thread1#####
Contador: 20000 (esperado: 20000)

```

Por que atomics. Com variáveis comuns, o compilador e o processador podem **reordenar** leituras e escritas para ganhar desempenho, desde que uma thread sozinha não perceba. Peterson depende de a escrita em interessado acontecer antes da leitura de vez e do interessado do outro. As funções `atomic_store` e `atomic_load` do C11 garantem essa ordem.

O que dá errado sem a ordem garantida. Processadores modernos têm *buffers de escrita*: uma escrita pode ficar guardada no núcleo por um instante antes de chegar à memória que os outros núcleos enxergam, enquanto a leitura seguinte já é feita. Se isso acontecer em Peterson, a sequência pode ser:

1. a thread 0 escreve `interessado[0] = 1`, mas a escrita fica no buffer do núcleo 0;
2. a thread 1 escreve `interessado[1] = 1`, que fica no buffer do núcleo 1;
3. as duas leem o `interessado` da outra na memória e veem 0;
4. as duas acham que a outra não quer entrar, e as duas entram.

As operações atômicas com ordenação *sequencialmente consistente* (o padrão de `atomic_load` e `atomic_store`) obrigam o processador a esvaziar o buffer antes de seguir, como se todas as operações de todas as threads acontecessem numa única ordem global. Esse acordo entre a linguagem e o hardware sobre a ordem das leituras e escritas se chama **modelo de memória**.

5.5 Espera ocupada

Enquanto espera, a thread fica girando no `while`: testa, testa, testa de novo. Isso é **espera ocupada** (*busy waiting*).

- ▶ A thread gasta CPU sem fazer trabalho útil.
- ▶ Com uma única CPU, ela pode até atrasar a thread que está na região crítica e precisa sair.
- ▶ Peterson também só funciona para duas threads; para mais, existem generalizações mais complicadas.

Seria melhor que a thread que não pode entrar **dormisse** e fosse **acordada** quando a região ficasse livre. É isso que os semáforos oferecem.

5.6 Quando girar compensa

A espera ocupada não é sempre ruim. Colocar uma thread para dormir e acordá-la depois exige uma **troca de contexto**, que custa alguns microssegundos. Se a região crítica é curtíssima e a outra thread está rodando em outro núcleo, girar alguns ciclos pode sair mais barato do que dormir. Por isso o núcleo do Linux usa spinlocks internamente, e implementações de mutex costumam girar um pouco antes de desistir e dormir.

Girar é ruim quando a espera pode ser longa, ou quando há uma CPU só: a thread que gira ocupa justamente o processador de que a outra precisa para terminar.

Inversão de prioridade

Com espera ocupada e prioridades, pode acontecer de uma thread de alta prioridade girar esperando uma trava que está com uma thread de baixa prioridade, que nunca recebe a CPU para soltá-la. Um problema desse tipo, com travas e prioridades, fez o robô Mars Pathfinder reiniciar repetidamente em Marte, em 1997, até ser corrigido por uma atualização enviada da Terra.

6 Semáforos

6.1 Dormir e acordar: o despertar perdido

Para evitar a espera ocupada, imagine duas operações do SO: `dormir()`, que suspende a thread, e `acordar(t)`, que a libera. Um consumidor ingênuo faria:

```
if (count == 0) dormir();      /* buffer vazio: dorme */
/* retira um item */
```

e o produtor, depois de inserir, `if (count == 1) acordar(consumidor);`. O defeito: o consumidor lê `count == 0` e, **antes** de chamar `dormir()`, perde a CPU. O produtor insere, vê `count == 1` e chama `acordar`, mas o consumidor ainda não está dormindo, e o aviso se perde. Quando o consumidor volta, dorme, e nunca mais é acordado. O produtor enche o buffer e também dorme. Os dois dormem para sempre.

A causa é a mesma de sempre: “testar a condição” e “dormir” precisam ser **uma operação só**. Em 1965, Edsger Dijkstra propôs uma variável que guarda os avisos não usados e faz o teste e a espera de forma indivisível: o **semáforo**. Os nomes originais das operações vêm do holandês: **P** (*proberen*, testar) e **V** (*verhogen*, incrementar).

6.2 Definição

Definição

Um **semáforo** é um contador inteiro que nunca fica negativo, com duas operações atômicas: `sem_wait(&s)` (também chamada *down* ou P): se `s > 0`, diminui 1 e segue; se `s == 0`, a thread **dorme** até que alguém faça `sem_post`. `sem_post(&s)` (também chamada *up* ou V): soma 1 e, se houver thread dormindo em `s`, acorda uma.

O teste, a alteração do contador e a decisão de dormir são feitos **atomicamente** pelo sistema: não há a corrida que estragou a variável de trava.

Analogia: estacionamento com placa de vagas

O valor do semáforo é o número na placa de vagas. Quem chega e vê 0 espera parado na entrada, sem ficar dando voltas no quarteirão (isso seria a espera ocupada). Quem sai, soma uma vaga na placa e libera o próximo da fila.

6.3 Como o semáforo funciona por dentro

Um semáforo é, no núcleo do SO, um contador e uma **fila** de threads que esperam:

```
sem_wait(s):                                sem_post(s):
[início da parte indivisível]                [início da parte indivisível]
se s.valor > 0:                               se a fila de s não está vazia:
    s.valor = s.valor - 1                     tira uma thread da fila
senão:                                       e a marca como pronta
    coloca esta thread na fila               senão:
    e a bloqueia (não gasta CPU)             s.valor = s.valor + 1
[fim da parte indivisível]                  [fim da parte indivisível]
```

A “parte indivisível” é garantida pelo núcleo com as ferramentas da Seção 4.6: um spinlock curtíssimo, só durante a atualização do contador e da fila. Ou seja, a espera ocupada não desapareceu: ela ficou restrita a poucas instruções, dentro do SO, em vez de durar a região crítica inteira. A espera longa, essa sim, vira bloqueio.

No Linux, a biblioteca ainda faz uma otimização: se o semáforo está livre, `sem_wait` resolve tudo com uma operação atômica, sem nem chamar o núcleo. Só quando é preciso dormir é que entra a chamada de sistema `futex`.

6.4 Semáforo como mutex

Um semáforo que começa em **1** funciona como uma **chave**:

```
sem_init(&mutex, 0, 1);    /* 1: a chave esta disponivel */

sem_wait(&mutex);           /* pega a chave (se outra thread tiver, dorme) */
contador = contador + 1;    /* regioao critica */
sem_post(&mutex);           /* devolve a chave */
```

Com isso, o `corrida.c` daria sempre 2000000. Um semáforo assim, que só vale 0 ou 1, é chamado de **semáforo binário** ou **mutex** (de *mutual exclusion*). Um semáforo que conta mais do que 1 é um **semáforo contador**.

6.5 Mutex e semáforo binário não são idênticos

Os dois começam livres e deixam passar uma thread por vez, mas há uma diferença conceitual:

- Um **mutex** tem **dono**: só a thread que travou pode destravar. Em `pthread_mutex_t` (com o tipo de verificação de erros), destravar um mutex que é de outra thread é um erro.
- Um **semáforo** não tem dono: qualquer thread pode fazer `sem_post`. Isso é o que permite usar `full` e `empty` como **sinais** entre threads: o produtor faz `sem_post(&full)` e quem faz `sem_wait(&full)` é outra thread.

Regra prática: para proteger uma região crítica, use um mutex; para contar recursos ou sinalizar eventos entre threads, use um semáforo. O `prod_cons.c` usa um semáforo binário como mutex por fidelidade ao exemplo clássico; com `pthread_mutex_t` funcionaria igual.

6.6 Funções POSIX

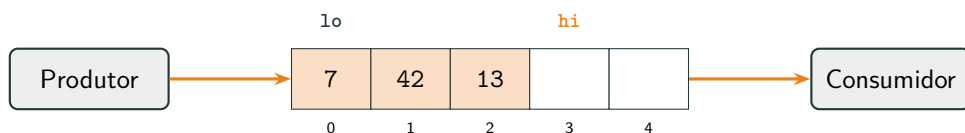
Função	O que faz
<code>sem_init(&s, 0, v)</code>	Cria o semáforo com valor inicial v . O 0 diz que ele é compartilhado entre threads do mesmo processo.
<code>sem_wait(&s)</code>	Espera e decrementa. Pode ser interrompida por sinal (EINTR); por isso o <code>prod_cons.c</code> tem o <i>helper</i> <code>wait_sem</code> , que tenta de novo.
<code>sem_post(&s)</code>	Incrementa e acorda uma thread que espera.
<code>sem_destroy(&s)</code>	Libera o semáforo quando ninguém mais o usa.

7 O problema do produtor-consumidor

7.1 O problema

Uma thread **produz** itens e os coloca num buffer de tamanho N ; outra **consome** itens desse buffer. As duas trabalham em ritmos diferentes. Precisamos garantir três coisas:

1. As duas nunca mexem no buffer ao mesmo tempo (**exclusão mútua**).
2. O produtor não insere com o buffer cheio (**espera por vaga**).
3. O consumidor não retira com o buffer vazio (**espera por item**).



O buffer é **circular**: hi é a próxima posição de escrita e lo a próxima de leitura. Os dois índices avançam com $(i + 1) \% N$, e depois da última posição voltam para a 0. Assim as posições são reaproveitadas e a ordem de chegada é mantida (FIFO).

7.2 Os três semáforos

Semáforo	Começa em	O que controla
<code>mutex</code>	1	Exclusão mútua : a chave do buffer.
<code>empty</code>	N	Disponibilidade : quantas vagas livres existem.
<code>full</code>	0	Disponibilidade : quantos itens prontos existem.

Dois problemas diferentes

O mutex responde “**posso mexer no buffer agora?**”. Os semáforos empty e full respondem “**tem o que eu preciso?**”. Confundir os dois é o erro mais comum neste problema.

7.3 Os passos de cada thread

Passo	Produtor	Consumidor
1	sem_wait(&empty): reserva uma vaga	sem_wait(&full): reserva um item
2	sem_wait(&mutex): pega a chave	sem_wait(&mutex): pega a chave
3	insere no buffer	retira do buffer
4	sem_post(&mutex): devolve a chave	sem_post(&mutex): devolve a chave
5	sem_post(&full): avisa que há mais um item	sem_post(&empty): avisa que há mais uma vaga

Regra para lembrar

Cada thread **espera o que consome** e **avisa o que cria**. O produtor consome vagas e cria itens; o consumidor consome itens e cria vagas.

7.4 Simulação com $N = 3$

Valores logo depois de cada operação terminar:

Evento	empty	full	count
início	3	0	0
produz A	2	1	1
produz B	1	2	2
consome A	2	1	1
produz C	1	2	2
produz D	0	3	3

Se o produtor tentar um quinto item, empty vale 0 e ele dorme em sem_wait(&empty), **sem segurar o mutex**, até o consumidor liberar uma vaga.

Repare que empty + full = N em todas as linhas. Isso vale sempre que nenhuma operação está em andamento; no meio de uma operação (depois do sem_wait e antes do sem_post), a soma pode ficar menor por um instante.

7.5 O programa prod_cons.c

```

1  /* Aula de SO: produtor-consumidor com buffer circular e semaforos POSIX.
2  *
3  * O que observar:
4  *   1. Uma thread PRODUZ numeros e outra os CONSOME, pelo mesmo buffer de N posicoes.
5  *   2. Tres semaforos, com papeis diferentes:

```

```

6  *      mutex (começa em 1): so uma thread por vez mexe no buffer  -> EXCLUSAO MUTUA
7  *      empty (começa em N): quantas vagas livres existem        -> DISPONIBILIDADE
8  *      full  (começa em 0): quantos itens prontos existem        -> DISPONIBILIDADE
9  *      3. Com buffer cheio, o produtor dorme em sem_wait(&empty).
10 *      Com buffer vazio, o consumidor dorme em sem_wait(&full).
11 *      4. A ORDEM importa: primeiro espera vaga/item, depois pega o mutex.
12 *      Na ordem inversa, as duas threads podem travar para sempre (deadlock).
13 *
14 * Semaforo em uma frase: um contador que nunca fica negativo.
15 * sem_wait: se o contador > 0, diminui 1 e segue; se for 0, DORME ate alguem fazer
    sem_post.
16 * sem_post: aumenta 1 e acorda quem estiver dormindo nele.
17 */
18 #include <assert.h>      /* assert: confere invariantes durante a execucao */
19 #include <errno.h>       /* errno, EINTR */
20 #include <pthread.h>     /* pthread_create, pthread_join */
21 #include <semaphore.h>   /* sem_t, sem_init, sem_wait, sem_post, sem_destroy */
22 #include <stdio.h>       /* printf, perror */
23 #include <stdlib.h>      /* rand, srand, exit */
24 #include <string.h>      /* strerror */
25 #include <time.h>       /* time, para a semente de rand */
26
27 #define N 20             /* capacidade do buffer */
28 #define TOTAL_ITENS 40   /* quantos itens o produtor gera antes de terminar */
29 static sem_t mutex, empty, full;
30 /* Buffer circular: hi = proxima posicao de escrita, lo = proxima de leitura,
31  * count = quantos itens estao guardados agora. */
32 static int buffer[N], lo, hi, count;
33
34 /* Funcoes sem_* devolvem -1 e preenchem errno em caso de erro. */
35 static void check_sem(int resultado, const char *operacao)
36 {
37     if (resultado == -1) { perror(operacao); exit(EXIT_FAILURE); }
38 }
39
40 /* sem_wait que tenta de novo se for interrompido por um sinal (EINTR). */
41 static void wait_sem(sem_t *sem)
42 {
43     int resultado;
44     do { resultado = sem_wait(sem); } while (resultado == -1 && errno == EINTR);
45     check_sem(resultado, "sem_wait");
46 }
47
48 /* pthread_* devolvem o codigo de erro (0 = sucesso). */
49 static void check_thread(int erro, const char *operacao)
50 {
51     if (erro != 0) {
52         fprintf(stderr, "%s: %s\n", operacao, strerror(erro));
53         exit(EXIT_FAILURE);
54     }
55 }
56
57 /* Chamadas somente com mutex adquirido. */
58 static void insert_item(int item)
59 {
60     assert(count < N);          /* empty garante que ha vaga */
61     buffer[hi] = item;
62     hi = (hi + 1) % N;         /* % N: depois da ultima posicao, volta para a 0 */
63     ++count;
64 }
65
66 static int remove_item(void)
67 {
68     assert(count > 0);          /* full garante que ha item */

```

```

69     int item = buffer[lo];
70     lo = (lo + 1) % N;          /* leitura também da a volta: FIFO circular */
71     --count;
72     return item;
73 }
74
75 static void *producer(void *arg)
76 {
77     (void)arg;
78     for (int i = 0; i < TOTAL_ITENS; ++i) {
79         int item = rand() % 100; /* Apenas o produtor usa rand. */
80         wait_sem(&empty);        /* 1. reserva uma VAGA (dorme se o buffer estiver cheio) */
81         wait_sem(&mutex);        /* 2. pega a chave do buffer */
82         insert_item(item);       /* 3. região crítica: mexe em buffer, hi e count */
83         printf("Produzido: %d | ocupacao: %d/%d\n", item, count, N);
84         check_sem(sem_post(&mutex), "sem_post mutex"); /* 4. devolve a chave */
85         check_sem(sem_post(&full), "sem_post full");    /* 5. avisa: ha mais um ITEM */
86     }
87     return NULL;
88 }
89
90 static void *consumer(void *arg)
91 {
92     (void)arg;
93     for (int i = 0; i < TOTAL_ITENS; ++i) {
94         wait_sem(&full);         /* 1. reserva um ITEM (dorme se o buffer estiver vazio) */
95         wait_sem(&mutex);        /* 2. pega a chave do buffer */
96         int item = remove_item(); /* 3. região crítica: mexe em buffer, lo e count */
97         /* count e lido dentro da mesma região que protege suas escritas. */
98         printf("Consumido: %d | ocupacao: %d/%d\n", item, count, N);
99         check_sem(sem_post(&mutex), "sem_post mutex"); /* 4. devolve a chave */
100        check_sem(sem_post(&empty), "sem_post empty"); /* 5. avisa: ha mais uma VAGA */
101        /* Processamento do item, se necessario, pode ocorrer aqui fora do lock. */
102    }
103    return NULL;
104 }
105
106 int main(void)
107 {
108     srand((unsigned)time(NULL));
109     /* sem_init(semaphore, 0 = compartilhado entre threads do processo, valor inicial) */
110     check_sem(sem_init(&mutex, 0, 1), "sem_init mutex"); /* buffer começa livre */
111     check_sem(sem_init(&empty, 0, N), "sem_init empty"); /* todas as N vagas livres */
112     check_sem(sem_init(&full, 0, 0), "sem_init full");    /* nenhum item ainda */
113     pthread_t produtor, consumidor;
114     check_thread(pthread_create(&produtor, NULL, producer, NULL), "pthread_create");
115     check_thread(pthread_create(&consumidor, NULL, consumer, NULL), "pthread_create");
116     /* join espera o termino; nao impede uma thread de terminar antes. */
117     check_thread(pthread_join(produtor, NULL), "pthread_join");
118     check_thread(pthread_join(consumidor, NULL), "pthread_join");
119     assert(count == 0); /* tudo que foi produzido foi consumido */
120     check_sem(sem_destroy(&mutex), "sem_destroy mutex");
121     check_sem(sem_destroy(&empty), "sem_destroy empty");
122     check_sem(sem_destroy(&full), "sem_destroy full");
123     printf("Fim: %d itens produzidos e consumidos; buffer vazio.\n", TOTAL_ITENS);
124     return EXIT_SUCCESS;
125 }

```

```

$ ./prod_cons
Produzido: 29 | ocupacao: 1/20
Produzido: 51 | ocupacao: 2/20
Produzido: 63 | ocupacao: 3/20

```

```
...  
Fim: 40 itens produzidos e consumidos; buffer vazio.
```

A ordem das linhas “Produzido” e “Consumido” muda a cada execução, mas os itens sempre são consumidos na mesma ordem em que foram produzidos.

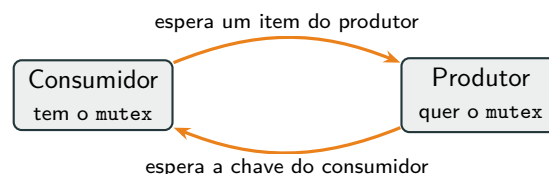
7.6 Por que a ordem dos `sem_wait` importa

Suponha que o consumidor pegasse a chave *antes* de esperar por item:

```
wait_sem(&mutex); /* pega a chave... */  
wait_sem(&full); /* ...e dorme com ela */
```

Com o buffer vazio:

1. O consumidor pega o mutex.
2. Dorme em `full`, **segurando a chave**.
3. O produtor tenta pegar o mutex para inserir e também dorme.



Deadlock

Cada thread espera algo que só a outra pode fornecer. Nenhuma avança, e o programa trava para sempre. Por isso, nas duas threads, **primeiro se espera a disponibilidade, depois se pega a chave**.

7.7 Vários produtores e vários consumidores

O mesmo código funciona com várias threads produtoras e várias consumidoras, sem mudança nenhuma. `empty` e `full` continuam contando vagas e itens no total, e o mutex garante que duas produtoras nunca escrevam na mesma posição `hi` ao mesmo tempo. É por isso que o mutex é indispensável: sem ele, duas produtoras poderiam ler o mesmo `hi`, gravar na mesma posição e avançar o índice duas vezes, perdendo um item, que é a condição de corrida da Seção 3 de novo.

8 Deadlock, inanição e livelock

8.1 As quatro condições do deadlock

Em 1971, Coffman e colegas mostraram que um deadlock só acontece se **quatro** condições valem ao mesmo tempo:

Condição	O que significa	No consumidor errado
Exclusão mútua	O recurso só pode ser usado por um de cada vez.	Só uma thread tem o mutex.
Posse e espera	Quem já tem um recurso espera por outro.	O consumidor tem o mutex e espera full.
Não preempção	Ninguém pode tomar um recurso à força.	O SO não tira o mutex do consumidor.
Espera circular	Existe um ciclo: A espera B, que espera A.	O consumidor espera o produtor, que espera o consumidor.

Basta quebrar **uma** delas para o deadlock ser impossível. A ordem correta do produtor-consumidor quebra a **posse e espera**: nenhuma thread espera por empty ou full segurando o mutex.

8.2 Como evitar na prática

- ▶ **Não dormir segurando uma trava** à espera de algo que outra thread só pode fazer com essa trava.
- ▶ **Ordem global de aquisição**: se um programa usa as travas A e B, todas as threads pegam sempre A antes de B. Assim não se forma o ciclo (quebra a espera circular).
- ▶ **Segurar travas pelo menor tempo possível**: fazer o trabalho demorado (cálculo, entrada e saída) fora da região crítica, como o comentário “processamento do item fora do lock” do `prod_cons.c`.

8.3 Inanição e livelock

Definição

Inanição (*starvation*): uma thread fica esperando indefinidamente, não porque o sistema travou, mas porque as outras sempre passam na frente. O sistema progride; aquela thread, não.

Livelock: as threads não estão bloqueadas, continuam executando, mas reagem umas às outras de um jeito que ninguém avança. A imagem clássica são duas pessoas num corredor que dão um passo para o mesmo lado ao mesmo tempo, repetidamente.

A condição 4 de Tanenbaum (ninguém espera para sempre) é justamente a ausência de inanição. Peterson a garante para duas threads; um mutex comum, em geral, não promete a ordem de chegada.

9 Monitores em Java

9.1 Por que monitores

Semáforos resolvem o problema, mas deixam toda a disciplina nas mãos do programador. É fácil:

- ▶ **esquecer** um `sem_post`, e alguma thread dorme para sempre;
- ▶ **trocar a ordem** de dois `sem_wait`, e o programa entra em deadlock;
- ▶ **proteger a coisa errada**, e a corrida continua.

Monitor

Um **monitor** é um objeto que junta os **dados compartilhados**, as **operações** que os acessam e a **exclusão mútua** dessas operações. Quem usa o objeto não precisa lembrar de pegar e devolver a chave: a linguagem faz isso.

Uma analogia comum: um banheiro com uma **chave única**. Quem tem a chave entra; quem chega depois espera na porta. Se quem está dentro descobre que falta papel, pode sair, **devolver a chave** e esperar ser avisado de que o papel chegou.

9.2 Origem: Hoare e Brinch Hansen

O conceito de monitor foi proposto por Per Brinch Hansen e por C. A. R. Hoare, no início dos anos 1970 (o artigo mais citado é o de Hoare, de 1974). A ideia tem duas partes:

1. **Exclusão mútua automática**: só uma thread por vez executa dentro do monitor. O compilador, e não o programador, insere o “pegar a chave” e o “devolver a chave”.
2. **Variáveis de condição**: filas onde uma thread espera até que uma condição sobre os dados do monitor se torne verdadeira. As operações são *wait* (esperar na condição, soltando o monitor) e *signal* (acordar uma thread que espera na condição).

Uma variável de condição **não guarda memória**, ao contrário do semáforo: um *signal* sem ninguém esperando simplesmente se perde. Por isso a thread sempre testa a condição, sobre os dados do monitor, antes de esperar. Não há despertar perdido porque teste e espera acontecem dentro do monitor, com a chave na mão.

Em Java, todo objeto pode ser um monitor, com **uma única** variável de condição implícita: `wait`, `notify` e `notifyAll` operam nela.

9.3 As ferramentas do Java

Recurso	O que faz
<code>synchronized</code>	Para executar um método <code>synchronized</code> , a thread precisa da chave (lock) do objeto. Só uma thread por vez fica dentro dos métodos <code>synchronized</code> de um mesmo objeto.
<code>wait()</code>	Solta a chave do objeto e dorme até ser avisada. Só pode ser chamado por quem tem a chave (senão, <code>IllegalMonitorStateException</code>).
<code>notify()</code>	Acorda uma thread que dormia em <code>wait()</code> nesse objeto. Ela não recebe a chave na hora: precisa disputá-la de novo.
<code>notifyAll()</code>	Acorda todas ; cada uma, ao conseguir a chave, confere a sua condição.
<code>Thread.sleep()</code>	Dorme por um tempo, mas não solta a chave .

9.4 Mailbox 3b: sem sincronização

O exemplo das pastas `examples/dia07/mailbox-3b` e `mailbox-3s` tem uma caixa de correio compartilhada por dois produtores e um consumidor. Dave envia “Hello, world.” e Bill envia “Hot dog!”. Na versão 3b não há nenhuma sincronização:

```

24 public class Mailbox3b {
25     // ESTADO COMPARTILHADO: os tres threads (2 produtores + 1 consumidor)
26     // mexem nestes campos ao mesmo tempo, sem nenhuma protecao.
27     private boolean youHaveMail;    // no longer public.
28                                     // state variables should be private!
29                                     // (true = ha uma mensagem esperando para ser lida)
30     private char [] message;        // a mensagem, guardada letra por letra
31     private final int MAXMESSAGE = 72;    // tamanho fixo do array
32     private final int MAXPROCESSTIME = 7; // pausa maxima (ms) entre duas letras
33
34
35     public Mailbox3b() {
36         youHaveMail = false;          // a caixa começa vazia
37         message = new char[MAXMESSAGE];
38     }
39
40     // Chamado pelos PRODUTORES. Nao e synchronized: Dave e Bill podem estar
41     // aqui dentro ao mesmo tempo. Este metodo inteiro e uma REGIAO CRITICA
42     // desprotegida.
43     public void storeMessage(String msg) {
44
45         // Copia a mensagem uma letra por vez, a partir da posicao 0.
46         for(int i = 0; i < msg.length(); i++) {
47             message[i] = msg.charAt(i);
48             try {
49                 // Pausa aleatoria de 0 a 6 ms entre as letras. Enquanto este
50                 // produtor dorme, o outro pode ser escalonado e escrever as
51                 // SUAS letras nas mesmas posicoes. Quanto maior MAXPROCESSTIME,
52                 // maior essa janela, e mais mensagens saem misturadas.
53                 Thread.sleep((int) (MAXPROCESSTIME * Math.random()));
54             }
55             catch (InterruptedException e) {} // interrupcao ignorada
56         }
57         // Completa o resto do array com espacos (apaga sobras de mensagens maiores).

```

```
58         for(int i = msg.length(); i < MAXMESSAGE; i++)
59             message[i] = ' ';
60         youHaveMail = true;           // avisa que ha correio (mas ninguem e acordado:
61                                     // o consumidor so descobre quando perguntar de novo)
62     }
63
64
65     // Chamado pelo CONSUMIDOR, a cada 20 ms. Tambem nao e synchronized:
66     // pode ler o array enquanto um produtor ainda esta escrevendo nele.
67     public String retrieveMessage() {
68
69         if(youHaveMail) {
70             // Le as 72 letras do jeito que estiverem AGORA, talvez pela metade.
71             String msg = new String(message,0, 72);
72             youHaveMail = false;      // marca a caixa como vazia
73             return (msg );
74         }
75         else {
76             // Caixa vazia: em vez de esperar, volta de maos vazias.
77             // (Na versao 3s, o consumidor DORME com wait() ate chegar correio.)
78             return "How sad, no mail ..";
79         }
80     }
81 }
```

storeMessage escreve a mensagem **letra por letra** no mesmo array, com uma pequena pausa entre as letras. Se os dois produtores chamam ao mesmo tempo, as letras se misturam. Saída real:

```
My name is, Bill. I say, Hello, worl
My name is, Bill. I say, Hot dog!    .
My name is, Dave. I say, Het dog!
```

Bill “disse” a frase de Dave, a mensagem foi cortada, e “Het dog!” é metade de cada um. É a condição de corrida de novo, agora em Java. O Thread.sleep entre as letras só aumenta a janela em que a outra thread pode entrar: por isso aumentar MAXPROCESSTIME deixa o problema mais visível.

9.5 Mailbox 3s: com monitor

```

24 public class Mailbox3s {
25     // Mesmo estado da 3b, agora protegido pelo monitor deste objeto.
26     private boolean youHaveMail = false;    // ha mensagem nao lida?
27     private char [] message;
28     private final int MAXMESSAGE = 72;
29     private final int MAXPROCESSTIME = 7;
30
31     public Mailbox3s() {
32         youHaveMail = false;
33         message = new char[MAXMESSAGE];
34     }
35
36     // synchronized: antes de executar, a thread pega a chave da caixa.
37     // Se outro produtor (ou o consumidor) estiver dentro, esta espera na porta.
38     public synchronized void storeMessage(String msg) {
39
40         for(int i = 0; i < msg.length(); i++) {
41             message[i] = msg.charAt(i);
42             try {
43                 // A mesma pausa da 3b. Mas Thread.sleep NAO solta a chave:
44                 // o produtor dorme entre as letras SEGURANDO o monitor, e
45                 // por isso ninguem consegue escrever no meio da mensagem.
46                 Thread.sleep((int) (MAXPROCESSTIME * Math.random()));
47             }
48             catch (InterruptedException e) {}
49         }
50         for(int i = msg.length(); i < MAXMESSAGE; i++)
51             message[i] = ' ';
52         youHaveMail = true;
53         // Acorda UMA thread que esteja em wait() neste objeto (o consumidor).
54         // Ela nao entra na hora: so depois que este metodo terminar e a
55         // chave for devolvida. (Aqui nao ha "enquanto cheia: wait()":
56         // e por isso que uma mensagem nao lida pode ser sobrescrita.)
57         notify();
58     }    // ao sair do metodo synchronized, a chave e devolvida
59
60
61     public synchronized String retrieveMessage() {
62
63         // Sem correio: wait() SOLTA a chave (para um produtor poder entrar
64         // e guardar) e coloca o consumidor para DORMIR, sem gastar CPU.
65         // Ao ser acordado, ele precisa pegar a chave de novo e, por causa
66         // do while, CONFERE outra vez se ha correio antes de seguir.
67         // (Com if, seguiria mesmo num despertar espurio ou se outra thread
68         // tivesse mudado o estado antes dele.)
69         while (youHaveMail == false) {
70             try {
71                 wait();
72             }
73             catch (InterruptedException e) {}
74         }
75
76         if(youHaveMail) {    // now always true at this point!
77             // (o while acima garante: aqui sempre ha correio)
78             String msg = new String(message,0, MAXMESSAGE);
79             youHaveMail = false;    // caixa vazia de novo
80             return (msg );
81         }
82         else
83             return "How sad, no mail.";    // nunca acontece nesta versao

```

```
84     }  
85  
86  
87  
88  
89 }
```

- ▶ Os dois métodos são *synchronized*: enquanto um produtor escreve, o outro produtor e o consumidor esperam pela chave. As mensagens saem **íntegras**.
- ▶ O consumidor, se não há mensagem, chama `wait()`: solta a chave (para que um produtor possa entrar) e dorme. Não fica perguntando “chegou?” o tempo todo.
- ▶ O produtor, ao terminar, chama `notify()` e acorda o consumidor.

9.6 Por que `while` e não `if`

O consumidor usa `while (youHaveMail == false) wait();`, e não um `if`. Motivos:

1. Quem acorda **não recebe a chave na hora**: precisa disputá-la. Enquanto isso, outra thread pode entrar e mudar a condição.
2. O Java permite **despertares espúrios**: uma thread pode sair do `wait()` sem que ninguém tenha chamado `notify`.

Com `while`, a thread confere a condição de novo ao acordar e só segue quando ela é verdadeira de fato.

9.7 Semântica de Hoare e semântica de Mesa

Quando uma thread faz *signal* e acorda outra, quem continua executando dentro do monitor?

- ▶ **Hoare** (*signal and wait*): a thread acordada entra **imediatamente** no monitor, e quem sinalizou espera. A condição que motivou o aviso ainda é verdadeira quando a acordada roda, e um `if` bastaria.
- ▶ **Mesa** (*signal and continue*), do sistema Mesa da Xerox, em 1980: quem sinalizou **continua**, e a acordada só volta para a fila da chave. Até ela conseguir entrar, outra thread pode ter mudado tudo.

Java, pthreads (`pthread_cond_wait`) e quase todas as linguagens atuais usam a semântica de **Mesa**, que é mais simples de implementar e combina bem com despertares espúrios. Consequência prática, que vale para toda a vida: **espere sempre dentro de um `while`**.

9.8 O limite da mailbox 3s

A 3s ainda pode perder mensagens

storeMessage não espera a caixa esvaziar. Se dois produtores guardam mensagens antes de o consumidor ler, a primeira é **sobrescrita**. A versão 3s garante que cada mensagem lida está **íntegra**, mas não garante que toda mensagem enviada seja **entregue**.

Uma caixa de uma posição sem perdas é exatamente o produtor-consumidor com $N = 1$. Com monitor, ficaria assim:

```
guardar(mensagem), synchronized:      retirar(), synchronized:
    enquanto cheia: wait()              enquanto vazia: wait()
    copia a mensagem                   copia a mensagem para o resultado
    cheia = verdadeiro                 cheia = falso
    notifyAll()                       notifyAll()
                                     devolve o resultado
```

Usa-se notifyAll porque, com dois produtores, um notify poderia acordar justamente o outro produtor (que veria a caixa cheia e voltaria a dormir) em vez do consumidor.

9.9 Uma caixa de correio correta, em Java

O pseudocódigo acima, escrito em Java e testado, está em `examples/dia07/caixa/`:

```
1  /**
2   * Aula de 7/10: caixa de correio de uma posicao, SEM perdas.
3   * E o produtor-consumidor com N = 1, escrito com monitor em vez de semaforos.
4   * Diferencas para a Mailbox3s: guardar() ESPERA a caixa esvaziar, e os dois
5   * metodos usam notifyAll(), porque ha dois produtores e um consumidor.
6   */
7  public class CaixaDeCorreio {
8      private String mensagem;          // a unica posicao
9      private boolean cheia = false;    // ha mensagem nao lida?
10
11     public synchronized void guardar(String msg) throws InterruptedException {
12         while (cheia)                  // caixa ocupada: solta a chave e espera
13             wait();
14         mensagem = msg;
15         cheia = true;
16         notifyAll();                   // acorda quem espera (o consumidor, em especial)
17     }
18
19     public synchronized String retirar() throws InterruptedException {
20         while (!cheia)                  // caixa vazia: solta a chave e espera
21             wait();
22         String msg = mensagem;
23         cheia = false;
24         notifyAll();                   // acorda quem espera (os produtores, em especial)
25         return msg;
26     }
27 }
```

O `TesteCaixa.java` da mesma pasta põe dois produtores para enviar 20000 mensagens cada e confere

o que chegou:

```
$ make run-caixa
Recebidas de Dave: 20000 | de Bill: 20000 (esperado: 20000 de cada)
```

Nenhuma mensagem se perde, e nenhuma sai misturada.

9.10 Ferramentas prontas do Java

Na prática, raramente se escreve `wait` e `notify` à mão. O pacote `java.util.concurrent` já oferece as estruturas desta aula, testadas e eficientes:

Classe	Corresponde a
Semaphore	Os semáforos do C (<code>acquire = sem_wait</code> , <code>release = sem_post</code>). É o que a simulação Produção de dinheiro usa.
ReentrantLock e Condition	Um monitor explícito, com quantas variáveis de condição se quiser (por exemplo, uma para “não cheio” e outra para “não vazio”).
ArrayBlockingQueue	O buffer limitado inteiro: <code>put</code> espera vaga e <code>take</code> espera item. É o produtor-consumidor pronto.
AtomicInteger	O contador atômico: <code>incrementAndGet</code> é o <code>atomic_fetch_add</code> do Java.

9.11 Semáforos ou monitores?

	Semáforo	Monitor
Exclusão mútua	Feita à mão, com um semáforo em 1	Automática
Esperar condição	Contadores (<code>empty</code> , <code>full</code>)	<code>wait</code> dentro de <code>while</code>
Guarda avisos?	Sim: o valor acumula os <code>post</code>	Não: <code>notify</code> sem ninguém esperando se perde
Onde fica a disciplina	Espalhada pelo código que usa	Concentrada no objeto
Erros típicos	Esquecer um <code>post</code> , trocar a ordem	Usar <code>if</code> , esquecer <code>notifyAll</code>

Os dois têm o mesmo poder: dá para implementar um com o outro. A escolha é de organização: o monitor deixa os erros mais difíceis de cometer, porque a chave é pega e devolvida pela linguagem.

10 Demonstração visual: Produção de dinheiro

10.1 A simulação

A simulação “Produção de dinheiro” mostra o produtor-consumidor acontecendo na tela. O **banco** é o produtor (cria notas) e o **carro-forte** é o consumidor (retira notas). Por baixo, ela usa threads Java e três semáforos, como o `prod_cons.c`.

```
$ make run-simulation
```

A janela precisa de JDK 25 ou mais novo e de Maven; na primeira execução, o Maven baixa o JavaFX pela internet.

Na tela	No <code>prod_cons.c</code>
10 posições	N
W (próxima escrita)	hi
R (próxima leitura)	lo
itens disponíveis	full
vagas livres	empty
“Produtor: Aguardando vaga”	dormindo em <code>sem_wait(&empty)</code>
“Consumidor: Aguardando item”	dormindo em <code>sem_wait(&full)</code>

10.2 Quatro experimentos

Para cada um, faça a previsão antes de olhar.

Experimento 1: ritmos iguais. Produtor e consumidor em 2 itens/s. O buffer fica oscilando com poucas notas, e W e R andam quase juntos. Há quase sempre vaga e quase sempre item: ninguém espera muito.

Experimento 2: produtor rápido. Produtor em 10, consumidor em 0,2. O buffer enche e aparece “Produtor: Aguardando vaga: buffer cheio”. No C, o produtor está dormindo em `wait_sem(&empty)` com `empty = 0`. Ele **não** segura o mutex, e por isso o consumidor continua conseguindo retirar.

Experimento 3: consumidor rápido. Produtor em 0,2, consumidor em 10. O buffer esvazia e aparece “Consumidor: Aguardando item: buffer vazio”. O consumidor dorme em `wait_sem(&full)` com `full = 0`; cada nota nova faz `sem_post(&full)` e o acorda.

Experimento 4: a volta do buffer e a pausa. Com ritmos intermediários, W chega à posição 9 e volta para a 0: é o $(hi + 1) \% N$. Pausando, itens disponíveis mais vagas livres dá sempre 10: é o $empty + full = N$.

O que a simulação mostra

O mutex garante que ninguém atrapalha. empty e full garantem que ninguém trabalha sem ter o que precisa.

11 Atividade prática

11.1 O que fazer

Abra `examples/dia07/prod_cons_atividade.c`. É o `prod_cons.c` com seis lacunas:

```
sem_init(&mutex, 0, 1);
sem_init(&empty, 0, /* 1: vagas no inicio */);
sem_init(&full, 0, /* 2: itens no inicio */);

/* produtor */          /* consumidor */
wait_sem(/* 3 */);      wait_sem(/* 5 */);
wait_sem(&mutex);        wait_sem(&mutex);
insert_item(item);      item = remove_item();
sem_post(&mutex);        sem_post(&mutex);
sem_post(/* 4 */);      sem_post(/* 6 */);
```

O mutex já está pronto e a ordem das linhas não deve mudar. As perguntas que guiam:

- ▶ Quantas vagas e quantos itens existem quando o programa começa?
- ▶ Antes de mexer no buffer, o que cada thread precisa ter?
- ▶ Depois de mexer, quem precisa ficar sabendo, e de quê?

11.2 Compilar e testar

```
$ gcc -std=c11 -Wall -Wextra -Wpedantic -Werror \
    examples/dia07/prod_cons_atividade.c -o prod-cons-aluno -pthread
$ ./prod-cons-aluno
...
Fim: 40 itens produzidos e consumidos; buffer vazio.
```

Antes de preencher, o esqueleto não compila. Isso é proposital.

11.3 Se der errado

Sintoma	O que significa
O programa para de imprimir e não termina (saia com Ctrl + C)	Alguma thread dorme esperando um semáforo que ninguém vai liberar.
Assertion 'count < N' failed	O produtor inseriu sem haver vaga: empty não está diminuindo de verdade.
Assertion 'count > 0' failed	O consumidor retirou sem haver item: full está maior do que deveria.

11.4 Perguntas para responder em dupla

1. Por que o produtor espera empty **antes** de pegar o mutex? O que poderia acontecer se pegasse o mutex primeiro?
2. Se o seu programa travou ou abortou em alguma tentativa, qual lacuna estava errada e por que isso causou aquele sintoma?

12 Resumo

Conceito	Em uma frase
Thread	Fluxo de execução que divide a memória com as outras threads do processo.
Condição de corrida	O resultado depende da ordem em que as threads foram intercaladas.
Região crítica	Trecho que acessa o recurso compartilhado.
Exclusão mútua	No máximo uma thread por vez na região crítica.
Peterson	Exclusão mútua para duas threads, com espera ocupada.
Semáforo	Contador com wait e post; quem não pode seguir dorme.
Produtor-consumidor	mutex para exclusão mútua; empty e full para disponibilidade.
Instrução atômica	Lê e escreve a memória numa operação indivisível, mesmo com vários núcleos.
Spinlock	Trava em que quem espera fica girando; boa para esperas curtíssimas.
Deadlock	Cada thread espera algo que só a outra pode dar; exige as quatro condições de Coffman.
Inanição	Uma thread espera indefinidamente enquanto as outras passam na frente.
Variável de condição	Fila de espera dentro de um monitor; não guarda avisos.
Monitor	Dados, operações e exclusão mútua no mesmo objeto; wait e notify para esperar condições.

As duas aulas em uma frase

Processos isolam a memória e precisam de pipe para conversar. Threads compartilham a memória e precisam de sincronização para não se atrapalhar.

13 Exercícios de fixação

1. Por que o resultado de `corrida.c` muda a cada execução?
2. Escreva uma intercalação dos três passos de `contador++` em duas threads que dê o resultado **correto** (2) e outra que dê o resultado errado (1).
3. No produtor-consumidor com $N = 5$, depois de três produções e um consumo completos, quanto valem `empty`, `full` e `count`?
4. Por que Peterson não obriga as threads a se alternarem (uma vez cada)?
5. E se o semáforo `mutex` fosse iniciado com 0 em vez de 1?
6. No produtor, trocar a ordem dos dois `sem_post` (avisar `full` antes de devolver o `mutex`) causa deadlock?
7. Por que `wait()` só pode ser chamado dentro de um método `synchronized`?
8. Na mailbox completa com dois produtores e um consumidor, por que usar `notifyAll()` e não `notify()`?
9. Se o consumidor da mailbox 3s usasse `Thread.sleep(100)` em vez de `wait()` dentro do `while`, o que aconteceria?
10. Das 20 intercalações possíveis de dois `contador++`, quais dão o resultado certo? Por quê?
11. Com `-O2`, o `contador = contador + 1` vira uma única instrução `addl`. Por que a corrida continua?
12. Qual das quatro condições de uma boa solução a alternância estrita viola? Dê um exemplo.
13. No deadlock do consumidor que pega o `mutex` antes de esperar `full`, identifique as quatro condições de Coffman.
14. Num monitor com semântica de Hoare, o `while` antes de `wait` seria necessário? E em Java?
15. Por que um semáforo não sofre o problema do despertar perdido, mas o par `dormir()/acordar()` ingênuo sofre?

Respostas

1. Porque a ordem em que as threads são intercaladas depende do escalonador, da carga da máquina e de quantos núcleos estão livres, e isso muda a cada execução. Quantos incrementos se perdem depende de quantas vezes as leituras das duas threads caíram “no meio” uma da outra.
2. Correta: A lê 0, A calcula 1, A escreve 1, B lê 1, B calcula 2, B escreve 2. Errada: a da Seção 3.3, em que as duas leem 0 antes de alguma escrever.

3. `empty = 3`, `full = 2`, `count = 2`. Três produções levam `empty` de 5 a 2 e `full` de 0 a 3; o consumo devolve uma vaga e retira um item.
4. Porque a espera só acontece se o **outro** também estiver interessado. Se a outra thread não quer entrar, `interessado[outro]` é falso e a thread entra quantas vezes quiser seguidas.
5. Ninguém conseguiria pegar a chave: o primeiro `sem_wait(&mutex)` de qualquer thread dormiria para sempre, e o programa travaria logo no começo.
6. Não. O consumidor pode acordar um pouco antes e ter de esperar o `mutex` até o produtor devolvê-lo, mas o produtor devolve logo em seguida, sem esperar nada. Funciona; só muda um pouco o desempenho. O deadlock vem de **dormir** segurando a chave, não da ordem dos `post`.
7. Porque `wait()` precisa **soltar** a chave, e só quem tem a chave pode soltá-la. Além disso, testar a condição e começar a esperar precisa acontecer sem que outra thread mude a condição no meio. Fora de `synchronized`, o Java lança `IllegalMonitorStateException`.
8. Com `notify()`, o produtor que acabou de guardar pode acordar o **outro produtor**, que vê a caixa cheia e volta a dormir. O consumidor continua dormindo, e todos podem acabar esperando para sempre. Com `notifyAll()`, todos acordam e cada um confere a sua condição: o consumidor certamente acorda.
9. O consumidor dormiria **segurando a chave**, porque `sleep` não a solta. Nenhum produtor conseguiria entrar em `storeMessage` para colocar uma mensagem, e o consumidor ficaria acordando e dormindo para sempre com a caixa vazia.
10. Só AAABBB e BBBAAB, em que uma thread faz os três passos antes de a outra começar. Em qualquer outra, as duas leem o valor antes de alguma escrever, e uma escrita apaga a outra.
11. Porque uma instrução não é atômica entre núcleos: por dentro, o processador lê, soma e escreve, e outro núcleo pode ler no meio. Só o prefixo `lock` (gerado por `atomic_fetch_add`) torna a operação indivisível.
12. A condição 3: uma thread fora da região crítica não pode impedir outra de entrar. Se a thread 0 quer entrar duas vezes seguidas, precisa esperar a thread 1 entrar e sair, mesmo que a thread 1 nem queira entrar.
13. Exclusão mútua: só um tem o `mutex`. Posse e espera: o consumidor tem o `mutex` e espera `full`. Não preempção: ninguém tira o `mutex` dele. Espera circular: o consumidor espera um item do produtor, e o produtor espera o `mutex` do consumidor.
14. Com Hoare, a thread acordada entra imediatamente, com a condição garantida, e um `if` bastaria (desde que não houvesse despertares espúrios). Java usa a semântica de Mesa e permite despertares espúrios: o `while` é obrigatório.
15. Porque o semáforo guarda os avisos no seu valor: um `sem_post` feito antes do `sem_wait` não se perde, apenas soma 1, e o `sem_wait` seguinte passa direto. Além disso, o teste do valor e o bloqueio acontecem numa operação indivisível. No par ingênuo, o teste e o `dormir()` são separados, e o `acordar()` feito no meio não deixa registro.

Para saber mais

- ▶ Páginas de manual: `man 7 pthreads`, `man 3 pthread_create`, `man 3 sem_wait`, `man 3 sem_init`. Também em <https://man7.org/linux/man-pages/>.
- ▶ Contrato de `wait`, `notify` e despertares espúrios: documentação de `java.lang.Object`, em <https://docs.oracle.com/en/java/javase/21/docs/api/java.base/java/lang/Object.html>.
- ▶ TANENBAUM, A. S.; BOS, H. *Sistemas Operacionais Modernos*. 4. ed. São Paulo: Pearson, 2016. Seção 2.3: comunicação entre processos (condições de corrida, regiões críticas, Peterson, semáforos, monitores).
- ▶ SILBERSCHATZ, A.; GALVIN, P. B.; GAGNE, G. *Fundamentos de Sistemas Operacionais*. 9. ed. Rio de Janeiro: LTC, 2015. Capítulo 5: sincronização de processos.

Em outras edições, o assunto continua nos capítulos sobre sincronização, mas a numeração pode mudar.

- ▶ DOWNEY, A. B. *The Little Book of Semaphores*. Green Tea Press, livre, em <https://greenteapress.com/wp/semaphores/>. Dezenas de problemas de sincronização resolvidos com semáforos, em inglês.
- ▶ HERLIHY, M.; SHAVIT, N. *The Art of Multiprocessor Programming*. Morgan Kaufmann. Capítulo 2: exclusão mútua, com a prova completa de Peterson.
- ▶ GOETZ, B. et al. *Java Concurrency in Practice*. Addison-Wesley, 2006. Monitores, o pacote `java.util.concurrent` e os erros mais comuns em Java.